Adapting Tempo Analysis for Your Data

A practical guide to tuning librosa for non-standard audio

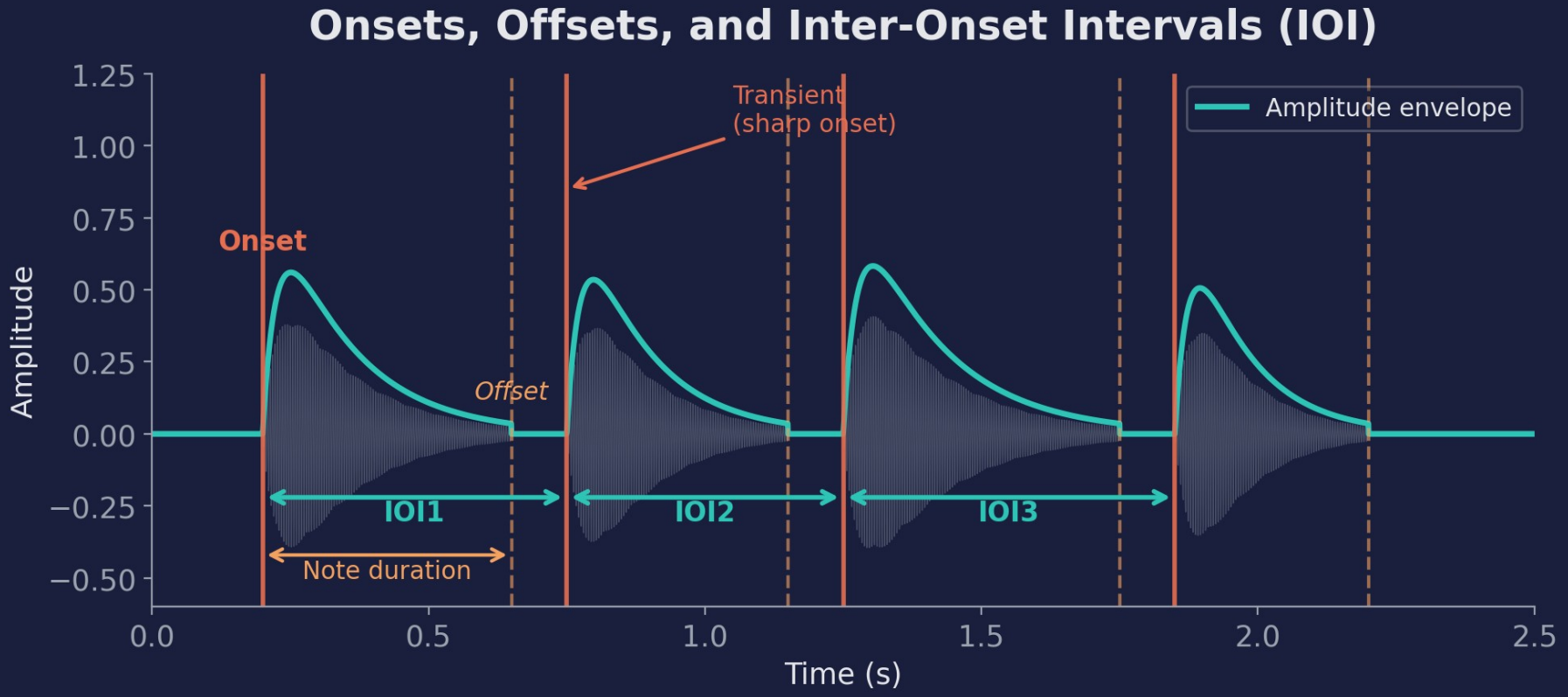
Benjamin D. Rieger
TU Dresden | CeTI Excellence Cluster



What is algorithmic tempo analysis?

- Using mathematical functions to predict the beat and tempo of a piece of music
- Autocorrelation, Beattracking, Dynamic Programming are the most common forms
- The shape and regularity of audio is measured in reference to the whole track and simulated or projected with multiple hypothetical grids

Anatomy of a Beat



IOI is the raw material for tempo: BPM = 60 / median(IOI)

When Defaults Fail

librosa's beat tracker is optimised for produced music with clear percussion.

A cappella singing	Soft onsets, no percussion, tempo drift
Field recordings	Background noise, variable dynamics
Speech / spoken word	Irregular rhythm, no clear beat
Ambient / drone	No transients, very slow or no pulse
Non-Western music	Unfamiliar meter, microtiming

If your audio doesn't sound like a pop song, you need to adapt.

What librosa Gives You

```
import librosa

y, sr = librosa.load('audio.wav', sr=None, mono=True)
env  = librosa.onset.onset_strength(y=y, sr=sr, hop_length=512)
bpm, beats = librosa.beat.beat_track(onset_envelope=env, sr=sr,
                                     hop_length=512, start_bpm=120)
```

Two core functions with tunable parameters:

`onset_strength()`
Spectral flux across mel bands. Params: sr, hop_length, n_fft, aggregate, n_mels

`beat_track()`
Dynamic-programming beat sequence. Params: start_bpm, tightness, trim, units

Knob 1: Pre-emphasis Filtering

Not a librosa parameter — you apply it to your audio before feeding it in.

```
y = librosa.effects.preemphasis(y, coef=0.97)
```

What it does

- Boosts high frequencies relative to low
- Makes consonants ('t', 'k', 's') louder relative to vowels
- Syllable boundaries become visible as amplitude peaks

When & how much

coef = 0.95–0.97
for singing with clear consonants

coef = 0.85–0.90
for humming or vowel-heavy audio

Skip entirely
for percussive audio (already sharp)

The Octave Problem

The single most common failure mode in tempo estimation:

Is this song at 60 BPM or 120 BPM?

Both are mathematically valid. Every beat at 60 BPM is also a beat at 120 BPM — you just skip every other one. Autocorrelation finds both.

Three strategies to resolve it:

1. **Tempo prior (soft bias)** — Multiply ACF by a Gaussian centred on your expected tempo. Gently discourages the wrong octave without forcing a result.
2. **Tactus correction (post-hoc)** — After estimation, check: is the result closer to 2× the expected? If yes, halve it. Simple and effective as a safety net.
3. **Grid-fit comparison** — Place a beat grid at BPM and BPM/2. Measure which grid aligns better with actual onset peaks. The better fit wins.

Knob 3: start_bpm & tightness

```
bpm, beats = librosa.beat.beat_track(onset_envelope=env, sr=sr,
             hop_length=512, start_bpm=120, tightness=100)
```

start_bpm

Initial tempo estimate for dynamic programming.

Default: 120 BPM (pop music)

Slow audio: 50–80
Fast audio: 140–180
Wrong start_bpm = garbage.

tightness

Controls beat rigidity. Higher = stricter; lower = flexible.

Default: 100

Steady music: 100–400
Rubato/expressive: 20–50
Too low = chases noise.

Knob 2: Onset Envelope Tuning

```
env = librosa.onset.onset_strength(y=y, sr=sr, hop_length=512)
env = librosa.util.normalize(env)                # scale to [0, 1]
```

hop_length

~12 ms at 44.1 kHz.
Time resolution.

Tuning:

Smaller = finer but noisier.
256 fast, 512–1024 slow.

smoothing

Moving average over N frames
suppresses noise.

Tuning:

5–10 for singing.
Less percussive, more ambient.

normalize

Puts all recordings on
same amplitude scale.

Tuning:

Almost always do this.
Prevents loud domination.

Building an ACF Estimator with Prior

```
def acf_tempo(onset_env, sr, hop, tempo_min, tempo_max, mu, sigma=5):
    """Pick tempo from ACF, biased toward expected range."""
    N = len(onset_env)
    acf = [np.dot(onset_env[:N-k], onset_env[k:]) / (N-k)
           for k in range(1, N//2)]
    bpms = 60 * sr / (hop * np.arange(1, len(acf)+1))
    mask = (bpms >= tempo_min) & (bpms <= tempo_max)

    # Gaussian prior — the key adaptation
    prior = np.exp(-0.5 * ((bpms[mask] - mu) / sigma)**2)
    score = acf[mask] * prior
    return bpms[mask][np.argmax(score)]
```

Copy-paste this. The two red lines are the entire prior mechanism.

Knob 3: start_bpm & tightness

```
bpm, beats = librosa.beat.beat_track(onset_envelope=env, sr=sr,
             hop_length=512, start_bpm=120, tightness=100)
```

start_bpm

The initial tempo estimate for the dynamic programming. The algorithm builds a beat sequence anchored around this value.

Default: 120 BPM (pop music)

If your audio is slow (lullabies, ambient), set this lower (50–80). If fast (EDM, punk), set higher (140–180). A wrong start_bpm can make the tracker drift entirely.

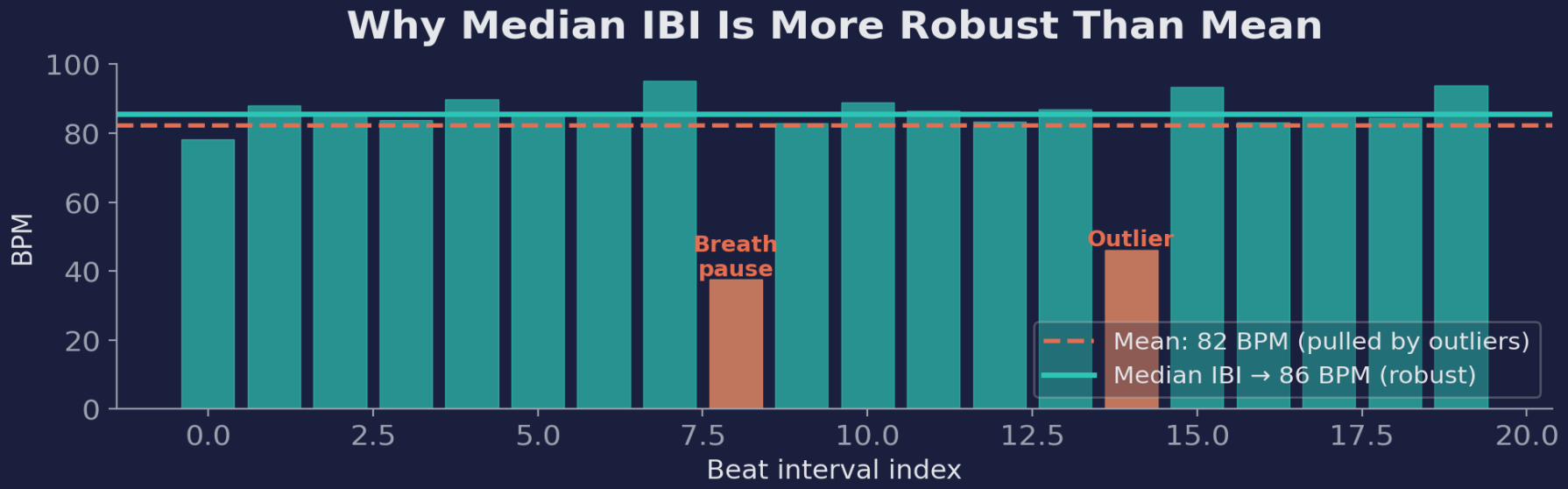
tightness

Controls how rigidly beats must follow the expected tempo. Higher = stricter metronome; lower = more flexible.

Default: 100

For steady music: 100–400 works well. For rubato / expressive performance: lower to 20–50 so the tracker can follow tempo changes. Too low and it chases noise.

Knob 4: Robust BPM from Beat Positions




```
beat_times = librosa.frames_to_time(beat_frames, sr=sr, hop_length=512)
ibis = np.diff(beat_times)                # inter-beat intervals
robust_bpm = 60.0 / np.median(ibis)       # median, not mean
```

Two lines. Dramatically better for anything with natural tempo variation.

Parameter Reference Card

Parameter	Default	Slow / soft audio	Fast / percussive
`pre-emphasis coef`	none	0.90 – 0.97	skip
`hop_length`	512	512 – 1024	256 – 512
`smoothing (frames)`	none	5 – 15	1 – 3
`start_bpm`	120	**50 – 80**	120 – 180
`tightness`	100	**20 – 50 (rubato)**	100 – 400
`ACF prior sigma`	n/a	5 – 10 (narrow)	15 – 30 (broad)
`BPM summary`	global	**median IBI**	global or median

Coral = changes that matter most.

Validation: How to Know It Works

You've tuned the parameters. How do you know the output is correct?

1. **Run multiple methods** — Use 2–3 estimators on same audio. If they agree, trust it.
2. **Report agreement** — Pearson r, ICC, MAD. ICC > .85 = good. Show in your paper.
3. **Spot-check by ear** — Listen with metronome overlay. Low-tech but catches systematic errors.
4. **Check your key effect** — Does your finding hold across all methods? If not, it may be an artefact.

Key Takeaways

1. librosa is a foundation, not a finished product. Adapt it.
2. Pre-emphasis is your best friend for vocal audio.
3. start_bpm and tightness matter most. Don't leave defaults.
4. The octave problem is real. Use priors, correction, or both.
5. Median IBI > global BPM for natural tempo variation.
6. Always validate. Multiple methods + listen.